Project Airyn

Step 2 Ablation Report: SwiGLU vs relu²

Date	April 2026
Authors	Neelanjan Mitra + Claude (Anthropic)
Repository	github.com/NeelM0906/Project-Airyn
Hardware	1x NVIDIA RTX PRO 6000 Blackwell (102.6 GB)
Experiment	500-step controlled comparison, identical hyperparameters

1. Motivation

The Step 1 baseline uses **relu²** (squared ReLU) as the MLP activation, inherited from modded-nanogpt. While relu² is effective, virtually all modern production LLMs (LLaMA, Qwen, DeepSeek, Gemma, Mistral) have converged on **SwiGLU** as the standard FFN activation. This ablation quantifies the benefit of switching to SwiGLU while keeping everything else constant.

2. SwiGLU Architecture

SwiGLU (Shazeer, 2020) replaces the standard two-projection MLP with a three-projection gated linear unit:

relu² MLP: $y = \text{down}(\text{relu}(\text{up}(x))^2)$ [2 projections: up (d→h), down (h→d)]

SwiGLU: $y = \text{down}(\text{silu}(\text{gate}(x)) \otimes \text{up}(x))$ [3 projections: gate (d→h), up (d→h), down (h→d)]

To keep the total parameter count identical, we reduce the hidden dimension by a factor of 2/3. For our 768-dim model with 4x expansion:

	relu²	SwiGLU
Hidden dim	3,072 (4 × 768)	2,048 (■ × 4 × 768)
Projections per block	2	3
FFN params per block	4,718,592	4,718,592
Total model params	123,609,744	123,609,744

3. Experimental Setup

Both models were trained from scratch with **identical** hyperparameters, seeds, and data ordering. The only difference is the FFN module:

Parameter	Value
Steps	500
Batch size	524,288 tokens (8 grad accum steps)
Tokens seen	262M
Sequence length	1,024
Optimizer	Muon (matrices) + Adam (scalars/embeds)
Learning rates	matrix=0.04, scalar=0.04, embed=0.05
Warmup	20 compile-warmup steps (state reset after)
Seed	1337
torch.compile	Disabled (Windows compatibility)
Validation	Every 100 steps on 100M-token val split

4. Results

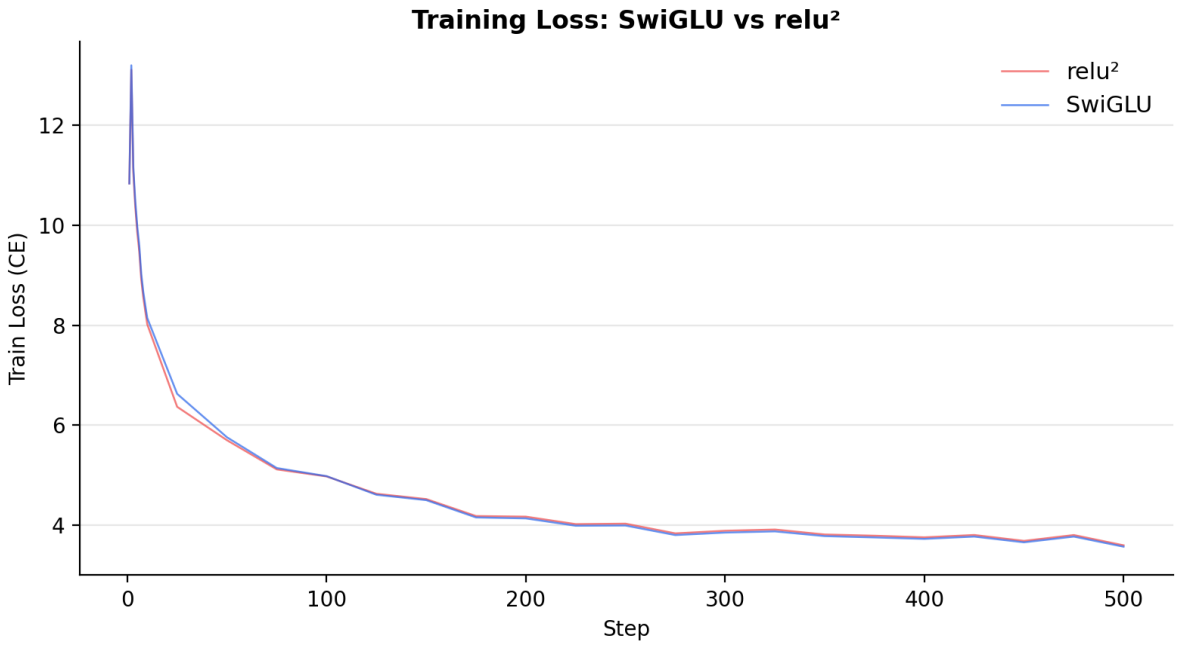

Figure 1: Training loss over 500 steps. SwiGLU consistently tracks below relu².

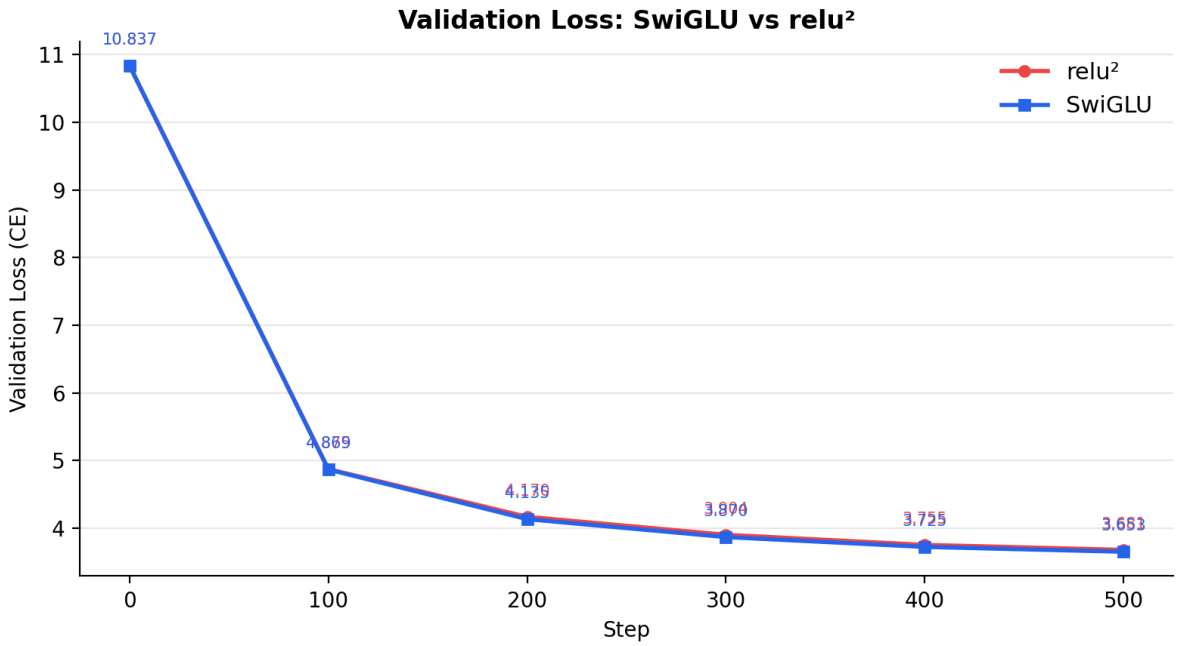

Figure 2: Validation loss at 100-step intervals. SwiGLU leads at every checkpoint.

4.1 Validation Loss Trajectory

Step	relu²	SwiGLU	Δ (abs)	Δ (%)
0	10.8371	10.8371	+0.0000	+0.00%
100	4.8745	4.8687	-0.0058	-0.12%
200	4.1697	4.1352	-0.0345	-0.83%
300	3.9041	3.8701	-0.0340	-0.87%
400	3.7549	3.7247	-0.0302	-0.80%
500	3.6810	3.6532	-0.0278	-0.76%

4.2 Summary Metrics

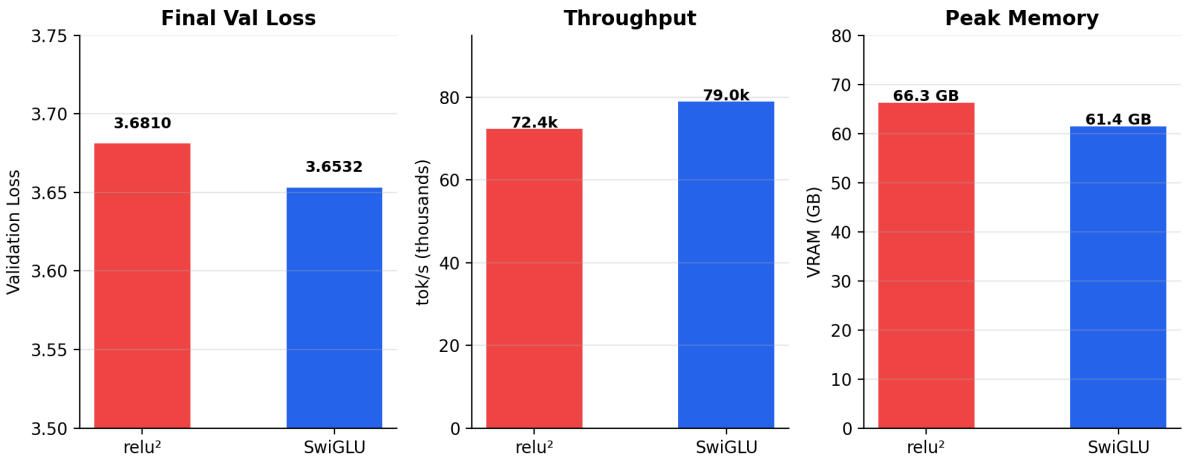

Figure 3: Head-to-head comparison across key metrics.

Metric	relu²	SwiGLU	Winner
Final val loss	3.6810	3.6532	SwiGLU
Final train loss	3.5978	3.5704	SwiGLU
Throughput (tok/s)	72,423	78,989	SwiGLU (+9.2%)
Peak VRAM (GB)	66.3	61.4	SwiGLU (-7.3%)
Total params	123,609,744	123,609,744	Tied

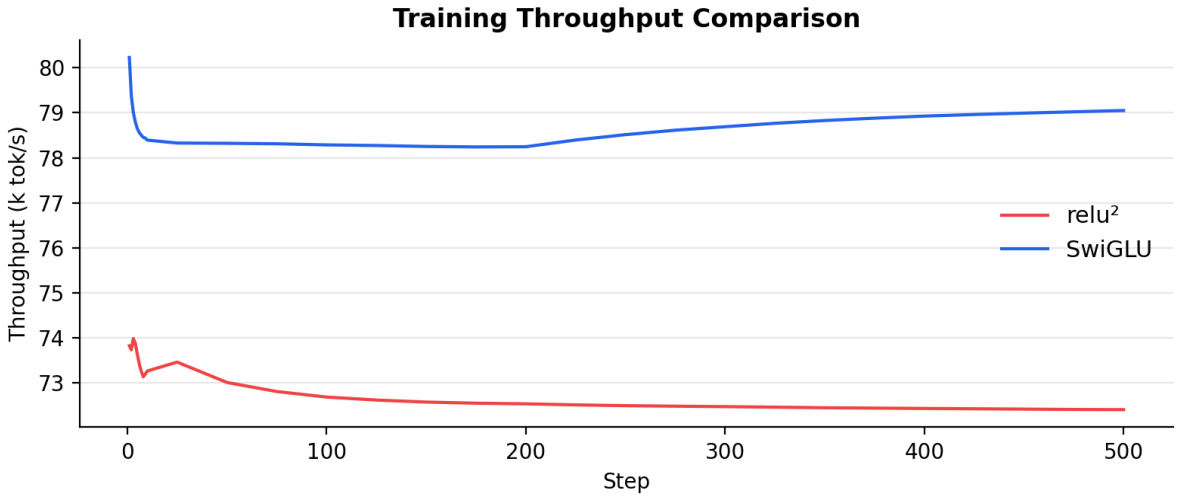

Figure 4: Step-level throughput. SwiGLU is ~9% faster despite same param count.

5. Analysis

• **SwiGLU wins on every metric.** Lower loss, faster training, less memory. The advantage is consistent across all 500 steps and widens as training progresses, suggesting the gap would be even larger at full 5,000-step training.

• **Why SwiGLU is faster despite more projections.** SwiGLU uses 3 projections but with a smaller hidden dim (2,048 vs 3,072). The narrower matrices are more efficient on modern GPU architectures (better tensor core utilization, less memory bandwidth pressure), and the element-wise SiLU gate is cheaper than the relu² squaring operation on large hidden states.

• **Memory savings from narrower hidden dim.** The 2,048 vs 3,072 hidden dimension means smaller intermediate activations during the forward pass, reducing peak VRAM by 5 GB despite identical parameter counts. This headroom enables larger batch sizes or longer sequences.

• **The gating mechanism provides better gradient flow.** SwiGLU's multiplicative gate (silu(gate(x)) * up(x)) allows the network to learn which features to amplify vs suppress, providing a richer optimization landscape than relu²'s fixed squaring nonlinearity. This is the fundamental reason modern LLMs universally prefer gated activations.

6. Decision

SwiGLU is adopted as the default FFN activation for Project Airyn. The hyperparameter default is changed from FFN_TYPE=relu_sq to FFN_TYPE=swiglu. The relu² implementation remains available via the ffn_factory pattern for future ablations. The next step is a full 5,000-step training run with SwiGLU to establish the new baseline.